

# Overview of Fully Homomorphic Encryption (FHE)

## Construction, Application and Acceleration

Yang Yang

Insight Computer Architecture Lab  
Department of Computer Science  
University of Virginia  
[yangyang@virginia.edu](mailto:yangyang@virginia.edu)

Fall 2026

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors
- 4 Mathematical Background
- 5 The CKKS Scheme
- 6 CKKS on GPUs

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors
- 4 Mathematical Background
- 5 The CKKS Scheme
- 6 CKKS on GPUs

# Where FHE sits: the post-quantum landscape

- NIST standardised its first **post-quantum** primitives in *August 2024* (FIPS 203-205):

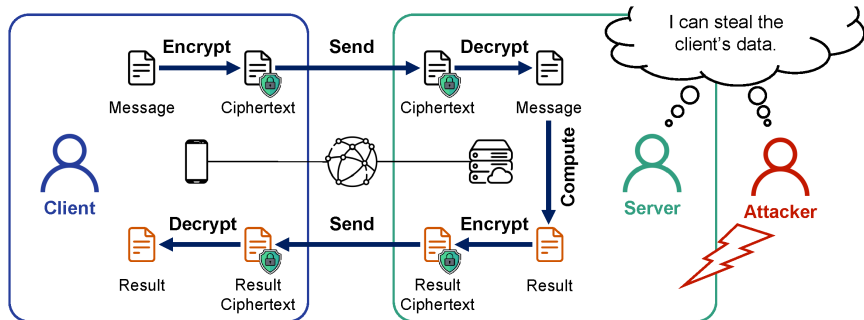
| Standard | What it is                                      | Name    |
|----------|-------------------------------------------------|---------|
| FIPS 203 | Module- <b>lattice</b> -based key encapsulation | ML-KEM  |
| FIPS 204 | Module- <b>lattice</b> -based digital signature | ML-DSA  |
| FIPS 205 | Stateless hash-based digital signature          | SLH-DSA |

## The connection

Homomorphic encryption is a **lattice-based** scheme too — it rests on the *same* hardness assumptions that NIST just standardised, and adds **computation on encrypted data** on top.

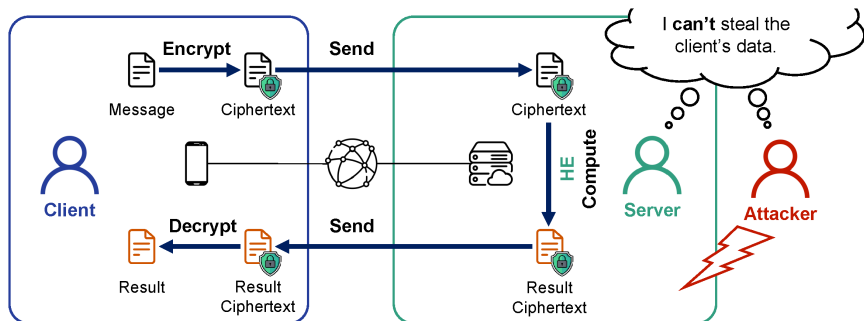
# The problem: the server sees your data

- In a conventional client-server round trip the server **decrypts** before it can compute.
- The plaintext therefore exists on the server: exposed to **data breach**, **side-channel attacks**, and **data misuse**.



# The fix: compute directly on ciphertexts

- HE lets the server evaluate the function **without decrypting** — the plaintext never exists outside the client.
- Computation can be offloaded to an untrusted cloud **without any leakage of private data**.



# The goal: computation you are not allowed to see

- Ordinary encryption hides  $x$  — but then the server can only *store* it, not compute on it.
- **Fully Homomorphic Encryption (FHE):** the server computes *directly on ciphertexts*; only the client can decrypt.

$$x \xrightarrow{\text{Enc}} \text{Enc}(x) \xrightarrow[\text{on the server}]{\text{Eval}(f)} \text{Enc}(f(x)) \xrightarrow[\text{client only}]{\text{Dec}} f(x)$$

The server sees only random-looking ciphertexts throughout.

# Homomorphic encryption, formally

A scheme is a tuple  $(\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ .

## Homomorphic property

For a function  $f$  in the supported class,

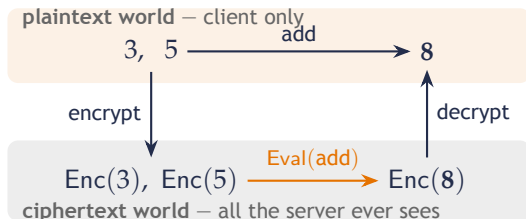
$$\text{Dec}\left(\text{Eval}(f, \text{Enc}(x_1), \dots, \text{Enc}(x_n))\right) = f(x_1, \dots, x_n).$$

- **Partially HE (PHE):** one operation only (e.g. RSA is multiplicative).
- **Somewhat HE (SWHE):** both  $+$  and  $\times$ , but only up to a bounded circuit *depth*.
- **Fully HE (FHE):** unbounded depth – enabled by *bootstrapping*.



# Two worlds: your data, and its ciphertexts

The client holds 3 and 5 and wants their sum. Two routes to the answer — **homomorphic** means *they agree*.



## Homomorphism

*textbook*

$$\varphi : G \rightarrow H$$

$$\varphi(x * y) = \varphi(x) \diamond \varphi(y)$$

*here*

$$\text{Dec} : \text{ciphertexts} \rightarrow \text{plaintexts}$$

$$\text{Dec}(c_1 \boxplus c_2) = \text{Dec}(c_1) + \text{Dec}(c_2)$$

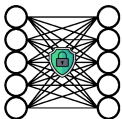
$\boxplus$  is what Eval does on the bottom row:  $\text{Dec}(\text{Enc}(3) \boxplus \text{Enc}(5)) = 3 + 5 = 8$ .

# A very short history

- **1978** RSA / ElGamal – partially homomorphic (one operation).
- **2009** Gentry's thesis – the *first* FHE, from lattices + bootstrapping. Proof of concept, but extremely slow.
- **2010** DGHV – a strikingly simple FHE over the *integers*.
- **2011-** practical families:
  - ▶ **BGV / BFV** – exact integer arithmetic.
  - ▶ **CKKS** (2017) – *approximate* real / complex arithmetic.
  - ▶ **TFHE** – very fast Boolean / bit operations.
- All modern schemes are built on **(Ring-)LWE** lattice hardness.

# Where FHE is already being used

- HE is emerging as a game-changer in **privacy-preserving computation**.



Private  
Machine Learning

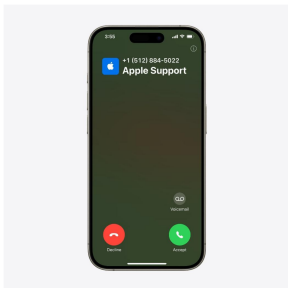


Private  
Healthcare

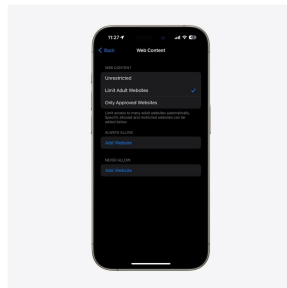


Private  
Data Analytics

...



Private Caller ID Lookup



Private Web Content Filtering

# Which scheme? The data type decides

| Scheme      | Data type in a message                           | Arithmetic                |
|-------------|--------------------------------------------------|---------------------------|
| BGV, BFV    | Finite integer $\mathbb{Z}_Q$                    | Integer (mod $Q$ )        |
| FHEW, TFHE  | Single bit $\{0,1\}$                             | Boolean (XOR, AND)        |
| <b>CKKS</b> | Real ( $\mathbb{R}$ ) / complex ( $\mathbb{C}$ ) | Approximate (fixed-point) |

- All three are built on (Ring-)LWE.
- **CKKS** is the one that speaks the native language of machine learning: real vectors and approximate arithmetic.
- That approximation is not a defect here – ML is already tolerant of it, and it buys a great deal of efficiency.

# CKKS: Message, plaintext and ciphertext



- **Encoding** is the *information-free* step: it maps your data into the algebraic object the scheme operates on (for CKKS, a polynomial in  $\mathcal{R}$ ). It hides nothing.
- **Encryption** is the step that provides *security*.

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping**
- 3 Lattices & Learning With Errors
- 4 Mathematical Background
- 5 The CKKS Scheme
- 6 CKKS on GPUs

# DGHV: a bootstrappable homomorphic scheme

Secret key: an odd integer  $p$ . Message  $m \in \{0, 1\}$ .

## Encrypt / decrypt (centered mod)

$c = \text{Enc}_p(m) = m + 2r + pq$ ,  $\text{Dec}_p(c) = (c \bmod p) \bmod 2$ ,  
small noise  $r$ , large random  $q$ ;  $a \bmod p \in (-p/2, p/2]$ .

- A ciphertext is a **noisy multiple of  $p$** :  $c \equiv \underbrace{2r + m}_{\text{error } e} \pmod{p}$
- Equivalently  $\text{Dec}_p(c) = \text{LSB}(c) \oplus \text{LSB}(\lceil c/p \rceil)$

# DGHV: what $+$ and $\times$ do to a ciphertext

Write  $c_i = m_i + 2r_i + p q_i$  and collect the *error*  $e_i = m_i + 2r_i$ , so that  $c_i \equiv e_i \pmod{p}$  with  $|e_i| \ll p$ . Its size is measured in bits:  $k_i := \log_2 |e_i|$ .

## Addition $\Rightarrow m_1 \oplus m_2$

$$\begin{aligned} c_1 + c_2 &= (m_1 + m_2) + 2(r_1 + r_2) + p(q_1 + q_2) \\ &\equiv \underbrace{e_1 + e_2}_{\text{new error}} \pmod{p} \end{aligned}$$

$$(e_1 + e_2) \bmod 2 = m_1 \oplus m_2 \quad \checkmark$$

$$|e| \leq |e_1| + |e_2| \Rightarrow k \approx \max(k_1, k_2) + 1$$

## Multiplication $\Rightarrow m_1 \wedge m_2$

$$\begin{aligned} c_1 c_2 &= (e_1 + p q_1)(e_2 + p q_2) \\ &= e_1 e_2 + p(e_1 q_2 + e_2 q_1 + p q_1 q_2) \\ &\equiv \underbrace{e_1 e_2}_{\text{new error}} \pmod{p} \end{aligned}$$

$$(e_1 e_2) \bmod 2 = m_1 \wedge m_2 \quad \checkmark$$

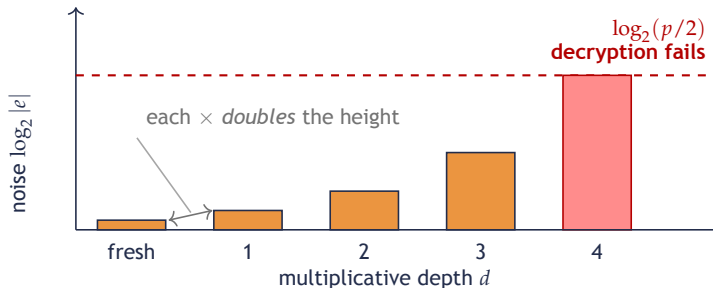
$$|e| \leq |e_1| |e_2| \Rightarrow k = k_1 + k_2 \quad (\text{doubles})$$

- Both operations are **plain integer arithmetic** on ciphertexts.
- $\{\oplus, \wedge\}$  is functionally complete, so *any* circuit is in principle evaluable – the sole obstacle is the **error**  $e$ , which every operation enlarges.



# The noise budget: why the depth is bounded

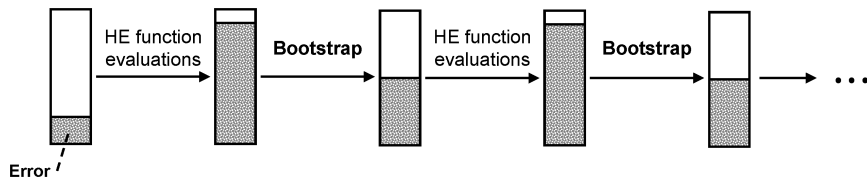
Decryption  $\text{Dec}_p(c) = (c \bmod p) \bmod 2$  recovers  $m$  **iff** the error stays inside the budget:  $|e| < p/2$ . With fresh noise  $\rho$  bits and  $p$  of  $\eta$  bits, depth  $d$  leaves  $k \approx \rho 2^d$ .



- **Add:**  $k \approx \max(k_i) + 1$  – costs **one bit**. **Mult:**  $k = k_1 + k_2$  – **doubles** the bit-length.
- Correctness needs  $\rho 2^d < \eta - 1$ , i.e.  $d \lesssim \log_2 \frac{\eta-1}{\rho}$  – **fixed and small** (somewhat homomorphic). Growing  $p$  buys only  $\log$  depth at linear cost; only **resetting** the noise escapes.

# Why bootstrapping is needed at all

- Every HE operation **accumulates error** in the ciphertext. Once the error reaches the top of the budget, the plaintext is lost.
- **Bootstrapping** (partially) **refreshes** that error.
- **FHE** = HE + bootstrapping  $\Rightarrow$  an *unlimited* number of function evaluations.

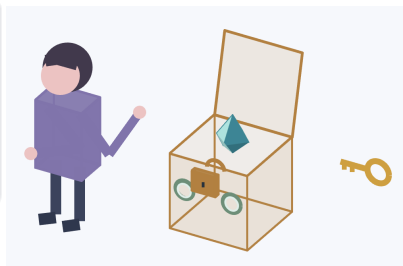


# Intuition: Alice's locked glovebox

## The picture (Gentry)

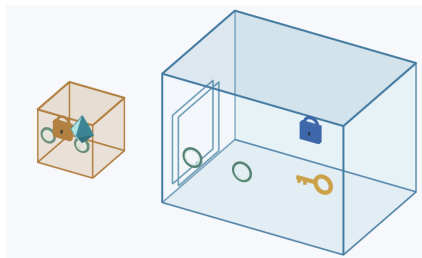
A *transparent, locked glovebox*: a one-way side door lets anyone *push data in*; only Alice's key opens the main lock; attached gloves let a worker *manipulate* the contents without opening it.

- glovebox = ciphertext, contents = plaintext;
- side door = public key  $pk$ , main lock = secret key  $s$ ;
- gloves = the **homomorphic property**: transform encrypted data meaningfully *without* learning it.



Alice holds the only key; the box is transparent but *locked*.

# Bootstrapping, in the glovebox picture



- The first box *jams* after a few operations – the noise budget runs out.
- So put box #1 **inside** box #2, which already holds an **encrypted copy of box #1's key**.
- Reaching through box #2's gloves, open box #1 *from the inside*: the contents end up in a **fresh** box, and work continues – recursively, forever.

## Animation

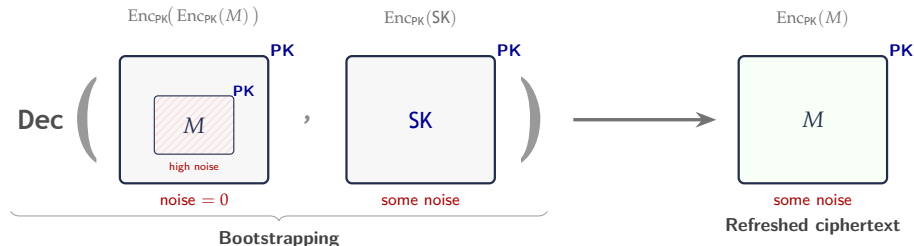
▶ [Play the bootstrapping animation](#)

# FHE: bootstrapping

## Gentry's nested boxes

Box *explodes* after a few operations. Put box #1 inside box #2 that already holds the key to box #1: open box #1 *from inside of* box #2 and keep working (recusively).

**Bootstrapping:** *homomorphically evaluate the decryption circuit* on a noisy ciphertext using an encrypted secret key gives a *refreshed* ciphertext with the noise reset.



A scheme that can evaluate its own decryption is **bootstrappable**.

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors**
- 4 Mathematical Background
- 5 The CKKS Scheme
- 6 CKKS on GPUs

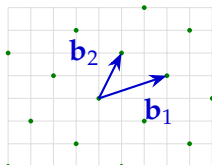
# Lattices

## Definition

Given linearly independent  $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{Z}^n$ , the lattice is all *integer* combinations

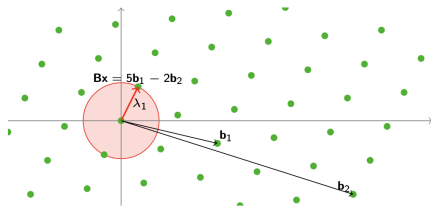
$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n c_i \mathbf{b}_i : c_i \in \mathbb{Z} \right\} \subset \mathbb{Z}^n,$$

a periodic grid;  $\lambda_1(\mathcal{L}) = \min_{\mathbf{v} \in \mathcal{L} \setminus \{0\}} \|\mathbf{v}\|$  is its shortest nonzero vector.



- Generating points is easy; *geometric* questions are hard.
- Hardness grows fast with dimension  $n$ : 2-D is solvable (Gauss), high- $n$  needs  $2^{\Omega(n)}$  time.

# Hard problem 1: Shortest Vector Problem (SVP)



**SVP:** find the shortest nonzero  $\mathbf{v} \in \mathcal{L} \setminus \{0\}$ ,

$$\lambda_1 = \min_{(c_1, \dots, c_n) \neq 0} \left\| \sum_i c_i \mathbf{b}_i \right\|.$$

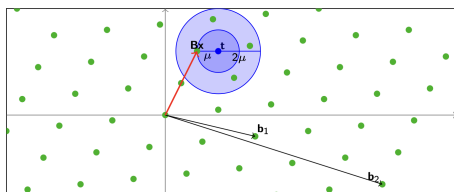
- With a *bad basis* the answer is a *huge* combination, e.g.  $5\mathbf{b}_1 - 2\mathbf{b}_2$  (huge – huge = tiny).
- 2-D: easy (Gauss). High  $n$ :  $2^{\Omega(n)}$ ; so far no poly-time algorithm is known.



# Hard problem 2: Closest Vector Problem (CVP)

A *decoding* problem: a target  $\mathbf{t}$  is a noisy point to be snapped onto the grid,

$$\mu = \text{dist}(\mathbf{t}, \mathcal{L}) = \min_{\mathbf{x} \in \mathbb{Z}^n} \|\mathbf{B}\mathbf{x} - \mathbf{t}\|.$$



- **Exact CVP** is NP-hard.
- $\text{CVP}_\gamma$ : return any lattice point within  $\gamma\mu$  of  $\mathbf{t}$ .
- “Decode a noisy point to the nearest lattice point” is *exactly* what LWE decryption does.

# Bridge: Short Integer Solution (SIS)

The *average-case* version of SVP and a collision-resistant hash.

$$\begin{array}{c} \boxed{\mathbf{A} \in \mathbb{Z}_q^{n \times m}} \\ \text{uniform (public)} \end{array} \times \begin{array}{c} \boxed{\mathbf{z}} \\ \text{short: } \|\mathbf{z}\| \leq \beta \end{array} \equiv \mathbf{0} \pmod{q}$$

- Given uniform  $\mathbf{A}$  ( $m \gg n$ , e.g.  $q \approx n^3$ ), find a nonzero *short*  $\mathbf{z}$  with  $\mathbf{Az} \equiv \mathbf{0}$ .
- Hash  $f_{\mathbf{A}}(\mathbf{x}) = \mathbf{Ax}$ ; a collision yields  $\mathbf{z} = \mathbf{x} - \mathbf{x}' \in \{-1, 0, 1\}^m$ ,  $\|\mathbf{z}\| \leq \sqrt{m}$ .

# Learning With Errors (LWE)

Fix a secret vector  $\mathbf{s} \in \mathbb{Z}_q^n$ . An LWE sample is

$$(\mathbf{a}, b) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e \bmod q), \quad \mathbf{a} \text{ uniform, } e \text{ small "noise"}.$$

- **Search-LWE:** given many samples, recover  $\mathbf{s}$ .
- **Decision-LWE:** distinguish samples from uniform random pairs.
- Both are believed **hard**, even for quantum computers.

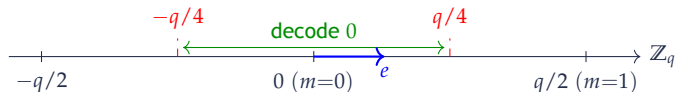
# Private-key encryption from LWE (Regev)

Secret key  $s \in \mathbb{Z}_q^n$ ; message  $m \in \{0,1\}$  is carried in the *most significant bit* by scaling it with  $\lfloor q/2 \rfloor$ .

## Scheme

$$\text{Enc}_s(m) : \mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n, e \leftarrow \chi; \quad c = (\mathbf{a}, b), \quad b = \langle \mathbf{a}, \mathbf{s} \rangle + e + m \lfloor q/2 \rfloor.$$

$$\text{Dec}_s(c) : v = b - \langle \mathbf{a}, \mathbf{s} \rangle \bmod q = \underbrace{e}_{\text{noise}} + m \lfloor q/2 \rfloor; \quad \hat{m} = 0 \text{ if } |v| < q/4, \text{ else } 1.$$



A ciphertext is an LWE sample with  $m \lfloor q/2 \rfloor$  added, so security is decision-LWE. Correct while  $|e| < q/4$ .

# Ring-LWE – the efficient version

Replace vectors / matrices by *ring elements*  $a, s, e \in \mathcal{R}_q$ :

$$(a, b) = (a, a \cdot s + e) \in \mathcal{R}_q^2.$$

- One RLWE sample packs  $N$  LWE samples “for free”  $\Rightarrow$  far smaller keys and ciphertexts.
- The multiply  $a \cdot s$  is a *polynomial* product – done fast with the NTT.
- RLWE is the foundation of **BGV**, **BFV**, and **CKKS**.

## This is our ciphertext

A CKKS ciphertext is essentially an RLWE sample whose noise also carries the message. Next: how CKKS puts real data into it.

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors
- 4 Mathematical Background**
- 5 The CKKS Scheme
- 6 CKKS on GPUs

# Roadmap of the math

DGHV proved FHE is *possible* (over the integers). The scheme we deploy – **CKKS**, real-number FHE – needs richer machinery, assembled from five classical tools:

- 1 **Modular arithmetic**  $\mathbb{Z}_q$  – the number system.
- 2 **Polynomial rings**  $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$  – where data live.
- 3 **Roots of unity** – the bridge to slots and NTT coordinates.
- 4 **CRT** – decomposes big moduli and polynomial rings.
- 5 **FFT / NTT** – moves between coefficient and evaluation views, making polynomial multiplication fast.

# Tool 1 – Modular arithmetic $\mathbb{Z}_q$

- $\mathbb{Z}_q = \{0, 1, \dots, q - 1\}$  with addition and multiplication *wrapping around* modulo  $q$ .
- Example ( $q = 7$ ):  $5 + 4 = 2$ ,  $3 \times 5 = 1 \pmod{7}$ .
- In FHE, ciphertext coefficients live in  $\mathbb{Z}_q$  with a *large* modulus  $q$  (hundreds of bits).



## Tool 2 – Polynomials and the ring $\mathcal{R}$

- $\mathbb{Z}[X]$ : polynomials with integer coefficients.
- We work *modulo*  $X^N + 1$  with  $N$  a power of two (e.g.  $N = 2^{16} = 65536$ ): whenever the degree reaches  $N$ , substitute  $X^N = -1$ .

$$\mathcal{R} = \mathbb{Z}[X]/(X^N + 1), \quad \mathcal{R}_q = \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(X^N + 1).$$

- An element is a polynomial  $a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1}$ , i.e., a **vector of  $N$  coefficients** in  $\mathbb{Z}_q$ :  $a = (a_0, a_1, \dots, a_{N-1})$ .

### Two operations

*Addition* is coefficient-wise. *Multiplication* is polynomial multiplication, then reduce by  $X^N + 1$ .

# Multiplication = negacyclic convolution

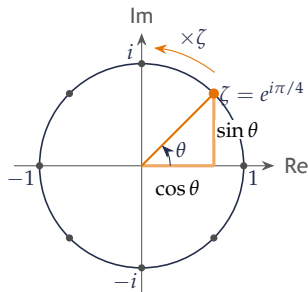
Multiplying in  $\mathcal{R}$  wraps high-degree terms back with a *sign flip* ( $X^N = -1$ ):

$$(a \cdot b)_k = \sum_{i+j=k} a_i b_j - \sum_{i+j=k+N} a_i b_j.$$

- This “negacyclic convolution” costs  $O(N^2)$  done naively – hopeless at  $N = 65536$ .
- Speeding it up is the whole reason the **NTT** exists.

## Tool 3 – Roots of unity

- A primitive  $2N$ -th root of unity satisfies  $\zeta^{2N} = 1$  and no smaller power equals 1.
- **Complex:**  $\zeta = e^{i\pi/N} \in \mathbb{C}$  is the unit-circle point at angle  $\pi/N$ ; each  $\times \zeta$  turns another  $\pi/N$ , so  $\zeta^{2N}$  is back at 1. Used to *encode* real data (FFT).
- **Modular version:**  $\omega \in \mathbb{Z}_q$  with  $\omega^{2N} = 1$  – exists iff  $q \equiv 1 \pmod{2N}$ , used to *multiply* polynomials (NTT).



Euler:  $e^{i\theta} = \cos \theta + i \sin \theta$   
( $N = 4$ )

### Key idea

Over  $\mathbb{C}$ , conjugate-paired evaluations define  $N/2$  CKKS slots; over  $\mathbb{Z}_q$ , the  $N$  evaluations are NTT coordinates. Multiplication becomes point-wise in either evaluation view.

# Roots of $X^N + 1$ – a concrete look ( $N = 4$ )

The roots of  $X^N + 1$  are the  $N$ -th roots of  $-1$ : the *odd*-indexed  $2N$ -th roots of unity,

$$x_k = e^{i(2k+1)\pi/N}, \quad k = 0, \dots, N-1.$$

**Example**  $N = 4$ : with  $\zeta = e^{i\pi/4}$  (a primitive 8-th root),

$$\zeta, \zeta^3, \zeta^5, \zeta^7 = \zeta, i\zeta, -\zeta, -i\zeta \quad (\zeta^2 = i, \zeta^4 = -1).$$

## Tool 4 – Chinese Remainder Theorem (CRT)

If  $q = q_1 q_2 \cdots q_L$  with pairwise-coprime  $q_i$ , then

$$\mathbb{Z}_q \cong \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \cdots \times \mathbb{Z}_{q_L}.$$

A number is equivalent to its tuple of residues, and arithmetic is independent per coordinate. CKKS implementations apply this idea along two storage axes:

### Axis 1 – integer CRT (RNS limbs)

Split a huge modulus  $q$  into machine-word primes  $q_i$  (30-60 bit). Each modular addition and multiplication becomes  $L$  independent small operations (massively parallel).

### Axis 2 – polynomial CRT (the NTT)

Within each NTT-friendly  $q_i \equiv 1 \pmod{2N}$ , factor  $X^N + 1$  and represent one polynomial by  $N$  finite-field evaluations. These are **NTT coordinates**.

# Worked example – CRT add and multiply

Take  $q = 35 = 5 \cdot 7$  and store each  $x \in \mathbb{Z}_{35}$  as its residues  $(x \bmod 5, x \bmod 7)$  – two small limbs.

|             | in $\mathbb{Z}_{35}$ | limb mod 5           | limb mod 7      |
|-------------|----------------------|----------------------|-----------------|
| $x$         | 17                   | 2                    | 3               |
| $y$         | 9                    | 4                    | 2               |
| $x + y$     | 26                   | $2 + 4 \equiv 1$     | $3 + 2 = 5$     |
| $x \cdot y$ | $153 \equiv 13$      | $2 \cdot 4 \equiv 3$ | $3 \cdot 2 = 6$ |

- Each limb did its own small add / multiply, and the results match:  $26 \leftrightarrow (1,5)$ ,  $13 \leftrightarrow (3,6)$ . ✓
- Recombine** only when needed:  $21 \leftrightarrow (1,0)$  and  $15 \leftrightarrow (0,1)$ , so  $(r_1, r_2) \mapsto 21r_1 + 15r_2 \bmod 35$ , e.g.  $(3,6) \mapsto 63 + 90 = 153 \equiv 13$ .

## Why it pays off

No limb ever touches the big modulus. In CKKS, a  $q$  of hundreds of bits becomes  $L$  word-size limbs, each processed independently and in parallel.

## Tool 5a – the DFT / FFT

- The **DFT** evaluates a degree- $< N$  polynomial at the  $N$  roots of unity – a multiply by the (Vandermonde) DFT matrix.
- The **FFT** does it in  $O(N \log N)$  by divide-and-conquer, instead of  $O(N^2)$ .
- **Cyclic convolution theorem** (modulo  $X^N - 1$ ): in the evaluation domain, multiplication is point-wise:

$$a \cdot b = \text{DFT}^{-1}(\text{DFT}(a) \odot \text{DFT}(b)).$$

The CKKS ring's negacyclic product uses the twist on the next slide.

- CKKS *encoding* uses a “special FFT” over  $\mathbb{C}$  (the canonical embedding) to map slots  $\leftrightarrow$  coefficients.

## Tool 5b – the NTT (FFT over a finite field)

- The **NTT** is the DFT with the complex root  $\zeta$  replaced by a *modular* root  $\omega \in \mathbb{Z}_q$  – exact integer arithmetic, no rounding.
- For the negacyclic  $X^N + 1$ , choose a primitive  $2N$ -th root  $\psi$  and set  $\omega = \psi^2$ , a primitive  $N$ -th root;  $\psi$  supplies the “twist.”
- **Fast multiplication** in  $\mathcal{R}_q$ :

$$a \cdot b = \text{NTT}^{-1}(\text{NTT}(a) \odot \text{NTT}(b)), \quad O(N \log N).$$

- High-performance implementations often keep ciphertext limbs in the NTT domain, so multiplication and key-switching use cheap point-wise products.

### Takeaway

The NTT is the single most-executed kernel in FHE – and the main target of hardware acceleration.



# The NTT *is* the CRT of polynomials

For a prime  $q \equiv 1 \pmod{2N}$ , a primitive  $2N$ -th root  $\psi \in \mathbb{Z}_q$  exists and  $X^N + 1$  splits into linear factors:

$$X^N + 1 = \prod_{i=0}^{N-1} (X - \psi^{2i+1}) \pmod{q}.$$

By CRT this is a ring isomorphism – and it *is* the NTT:

$$\mathcal{R}_q \cong \underbrace{\mathbb{Z}_q \times \cdots \times \mathbb{Z}_q}_{N \text{ NTT coordinates}}, \quad \text{NTT}(a) = (a(\psi), a(\psi^3), \dots, a(\psi^{2N-1})),$$

## Worked example (1/4) – $a \cdot b$ the schoolbook way

Take  $N = 4$  and  $q = 17$ , so we multiply in  $\mathcal{R}_{17} = \mathbb{Z}_{17}[X]/(X^4 + 1)$ :

$$a(X) = 1 + 2X + 3X^2 + 4X^3, \quad b(X) = 1 + X^2.$$

Multiply out, then fold the high terms back with  $X^4 = -1$ ,  $X^5 = -X$ :

$$\begin{aligned} a \cdot b &= 1 + 2X + 4X^2 + 6X^3 + 3X^4 + 4X^5 \\ &= (1 - 3) + (2 - 4)X + 4X^2 + 6X^3 \\ &= -2 - 2X + 4X^2 + 6X^3 \equiv 15 + 15X + 4X^2 + 6X^3 \pmod{17}. \end{aligned}$$

- In general every  $a_i$  meets every  $b_j$ :  $N^2 = 16$  products.
- **Goal:** get the same  $c = a \cdot b$  with only  $N = 4$  products.

## Worked example (2/4) – via the NTT

$q = 17 \equiv 1 \pmod{8}$ , and  $\psi = 2$  is a primitive 8-th root of unity ( $2^4 = 16 \equiv -1$ ). The roots of  $X^4 + 1$  are  $\psi, \psi^3, \psi^5, \psi^7 = 2, 8, 15, 9$ .

| evaluation point $x$                        | 2  | 8  | 15 | 9  |
|---------------------------------------------|----|----|----|----|
| $\hat{a} = a(x)$                            | 15 | 13 | 11 | 16 |
| $\hat{b} = b(x)$                            | 5  | 14 | 5  | 14 |
| $\hat{c} = \hat{a} \cdot \hat{b} \pmod{17}$ | 7  | 12 | 4  | 3  |

- Only  $N = 4$  point-wise products, e.g.  $15 \cdot 5 = 75 \equiv 7$  – and indeed the schoolbook  $c(2) = 109 \equiv 7$ . ✓
- $\text{NTT}^{-1}(7, 12, 4, 3) = (15, 15, 4, 6)$  recovers  $c$  exactly.

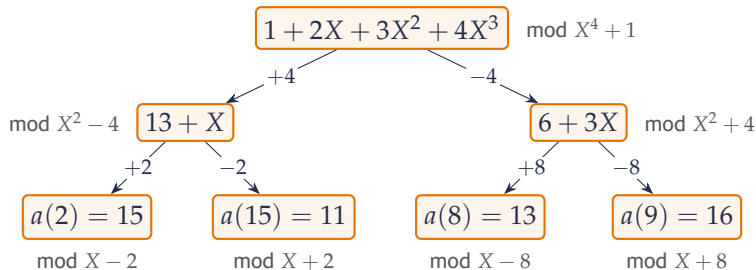
But

Evaluating  $N$  points one by one is still  $O(N^2)$ . The fast NTT walks the CRT factorization of  $X^N + 1$  instead.

# Worked example (3/4) – the fast NTT is a CRT tree

Mod 17,  $X^4 + 1$  splits in halves, and each split is one CRT step:

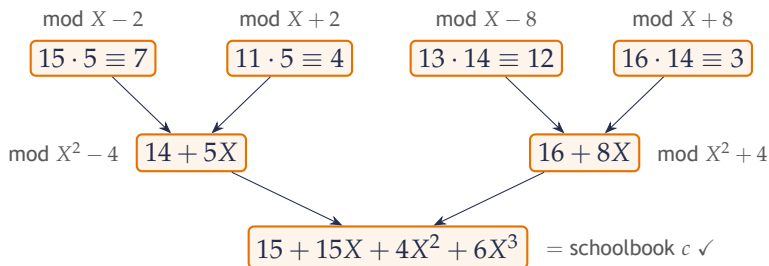
$$X^4 + 1 \equiv (X^2 - 4)(X^2 + 4) \equiv (X - 2)(X + 2)(X - 8)(X + 8).$$



- Arrow  $\pm w$ : reduce  $\ell + X^m h$  modulo  $X^m \mp w$ , giving  $\ell \pm w h$ . E.g.  $(1 + 2X) + 4(3 + 4X) = 13 + 18X \equiv 13 + X$ .
- Both children reuse  $w h$ : a **butterfly**  $(x, y) \mapsto (x + wy, x - wy)$ .
- $\log_2 N$  levels  $\times N/2$  butterflies  $= O(N \log N)$  – the Cooley-Tukey NTT. Leaves come out in bit-reversed order.

# Worked example (4/4) – multiply, then CRT back up

$b = 1 + X^2$  goes through the same tree, giving leaves 5, 5, 14, 14. Multiply leaf by leaf, then **undo** each split (CRT reconstruction):



- Inverse butterfly:  $\ell = (x + y)/2$ ,  $h = (x - y)/(2w)$ . E.g.  $(7, 4)$ ,  $w = 2$ :  $\ell = 11/2 \equiv 14$ ,  $h = 3/4 \equiv 5$ .
- This is the Gentleman-Sande inverse NTT; implementations merge all the  $1/2$ 's into one final scaling by  $N^{-1}$ .
- $2 \text{ NTTs} + N \text{ products} + 1 \text{ inverse NTT} = O(N \log N)$ .

# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors
- 4 Mathematical Background
- 5 The CKKS Scheme**
- 6 CKKS on GPUs

# CKKS at a glance

**CKKS** (Cheon-Kim-Kim-Song, 2017): *fixed-point arithmetic on encrypted vectors*. One ciphertext packs  $N/2$  real / complex numbers (“slots”), and one operation acts on all of them at once (SIMD).

## The round trip

$$\underbrace{z \in \mathbb{C}^{N/2}}_{\text{message}} \xrightarrow{\text{encode}} \underbrace{m(X) \in \mathcal{R}}_{\text{plaintext poly}} \xrightarrow{\text{encrypt}} \underbrace{(c_0, c_1)}_{\text{ciphertext}} \xrightarrow{+, \times, \text{rot}} (c'_0, c'_1)$$
$$(c'_0, c'_1) \xrightarrow{\text{decrypt}} m'(X) \xrightarrow{\text{decode}} z' \approx f(z)$$

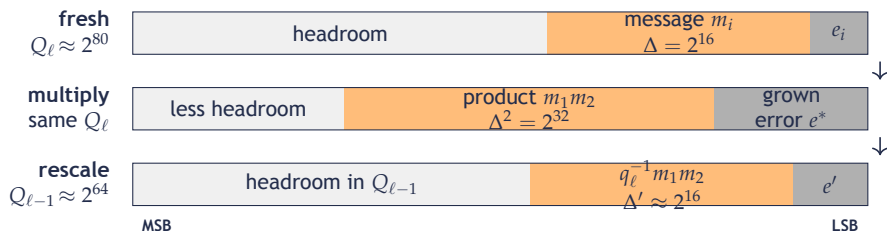
## What the hardware sees (typical $N = 2^{16}$ )

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| slots per ciphertext | $N/2 = 32,768$                                               |
| modulus $Q$          | $\sim 1500$ bits = $L \approx 30$ word-size primes           |
| one ciphertext       | $2 \times L \times N$ 64-bit words $\approx 30$ MB           |
| core kernels         | NTT, element-wise mod-mul/add, basis conversion, permutation |

# Fixed point with noise in the LSBs: scale $\Delta$ and levels

Decryption returns the scaled message plus small noise,

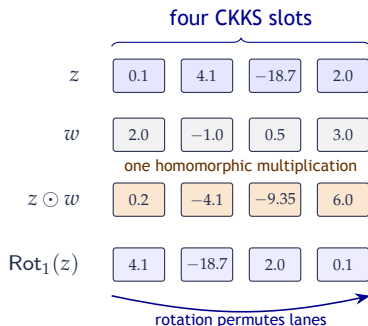
$u = [c_0 + c_1 \mathbf{s}]_{Q_\ell} = m_\Delta + e$  with  $m_\Delta \approx \Delta \cdot \text{Encode}(z)$ : message in the high bits, noise in the LSBs.



- A multiply squares the scale and grows the error. **Rescale** divides by a chain prime  $q_\ell \approx \Delta$ : the scale returns to  $\approx \Delta$  and the noisy LSBs are rounded away.
- Each rescale uses up one prime of the chain  $Q_L \supset \dots \supset Q_0$ , so a ciphertext has a fixed multiplicative **depth**; bootstrapping refills it.
- Unlike BFV/BGV (exact integers mod  $t$ ), the error is part of the answer – fine for ML and signal processing.



# Slots are SIMD lanes



$N = 8$  toy example: four independent complex slots.

- $X^N + 1$  has  $N$  complex roots in conjugate pairs, so a real polynomial holds  $N/2$  independent complex slots.
- One ciphertext  $+$  or  $\times$  performs the same operation in every slot: **CKKS SIMD**.
- Add / multiply never mix lanes; only a **rotation** moves data between slots.

A slot is not a coefficient. Encoding spreads every slot across all  $N$  plaintext coefficients; a slot becomes visible again only after decrypt + decode.

# Encoding: slots $\rightarrow$ polynomial $\rightarrow$ NTT form

$$z \in \mathbb{C}^{N/2} \xrightarrow{\text{special iFFT } (\mathbb{C})} \sigma^{-1}(z) \xrightarrow{\times \Delta, \text{ round}} m(X) \in \mathcal{R} \xrightarrow{\text{RNS, NTT } (\mathbb{Z}_{q_i})} \hat{m}$$

- $\sigma$  evaluates at the primitive  $2N$ -th roots of unity (the unit-circle points); encoding applies  $\sigma^{-1}$ , a **special FFT over  $\mathbb{C}$**  – data *packing*.
- Scale  $\Delta$  (e.g.  $2^{40}$ ) keeps fractional bits as integers; rounding is where the approximation starts.
- The **NTT over  $\mathbb{Z}_{q_i}$**  is a different, exact transform for fast *multiplication*; decode undoes only the first two steps.

## Tiny example: $N = 4$ , two slots, no scaling

$z = (1+2i, 3+4i)$ , extended by conjugates to the roots  $\zeta, \zeta^3, \zeta^5, \zeta^7$  of  $X^4 + 1$ , is hit by  $p(X) = 2 + \sqrt{2}X - X^2 + 2\sqrt{2}X^3$ :  $p(\zeta) = 1+2i$ ,  $p(\zeta^3) = 3+4i$ .

# Keys, encryption and decryption

## KeyGen (client, once)

Secret  $s$ : a *small* (ternary) polynomial. Public key  $pk = (-a s + e, a)$  with  $a$  uniform and  $e$  small. Evaluation keys:  $rlk \approx \text{Enc}_s(s^2)$  for relinearization and  $gk_r \approx \text{Enc}_s(s(X^{5^r}))$ , one per rotation distance  $r$ .

**Encrypt** a plaintext  $m$  (small random  $u, e_0, e_1$ ):

$$ct = (c_0, c_1) = (m + u \cdot pk_0 + e_0, u \cdot pk_1 + e_1).$$

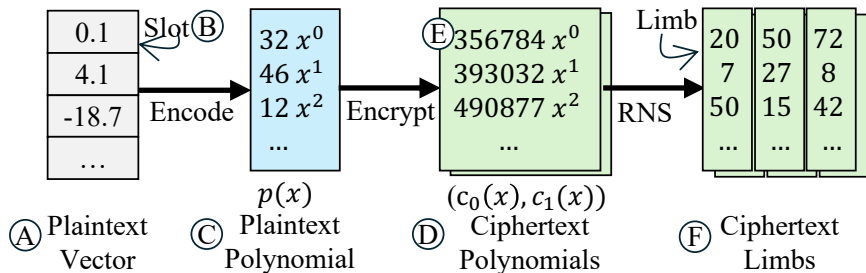
**Invariant and decrypt** (evaluate the linear form, then decode):

$$c_0 + c_1 \cdot s = m + e \pmod{Q}, \quad e \text{ small.}$$

## Who holds what

The client keeps  $s$ . The server gets ciphertexts plus  $pk$ ,  $rlk$  and the  $gk_r$  — each evaluation key is several ciphertexts' worth of data.

# Encoding and Encrypting Values in RNS-CKKS



**A/B** message  $z \in \mathbb{C}^{N/2}$ ; each entry  $z_j$  is a slot (SIMD lane).

**C** plaintext  $m(X) = \lfloor \Delta \sigma^{-1}(z) \rfloor$ : every slot spreads over all  $N$  coefficients.

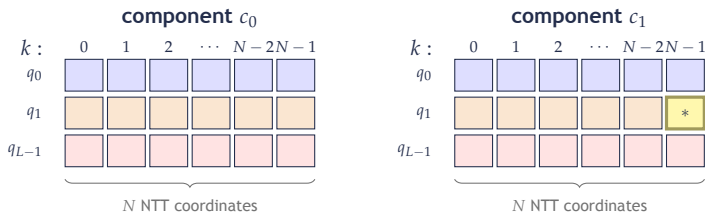
**D/E** ciphertext  $(c_0, c_1) \in \mathcal{R}_Q^2$ ,  
 $[c_0 + c_1 s]_Q = m + e$ . **E** is one coefficient in  $\mathbb{Z}_Q$ , not a slot.

**F** RNS limbs  $c_b^{(i)} = c_b \bmod q_i$ :  $2 \times L \times N$  machine words.

# Double-CRT: a ciphertext is a $2 \times L \times N$ tensor

Two exact CRT splits turn big-modulus polynomials into word-size cells:

$$\mathcal{R}_{Q_\ell}^2 \cong \left( \prod_{i=0}^{L-1} \mathcal{R}_{q_i} \right)^2 \cong \left( \prod_{i=0}^{L-1} \prod_{k=0}^{N-1} \mathbb{Z}_{q_i} \right)^2.$$



- **Row = limb:** one residue polynomial mod  $q_i$ . Add, multiply and NTT run independently per row.
- **Column = NTT coordinate:** element-wise ops are independent per column. The cell  $*$  is one 30-60-bit residue.
- **Level = number of rows:** a rescale deletes a row; the slot count stays  $N/2$ . Neither axis is a slot axis.

# Homomorphic operations and what they cost

| operation           | what happens to $ct = (c_0, c_1)$                                     | kernels                         | cost / depth                 |
|---------------------|-----------------------------------------------------------------------|---------------------------------|------------------------------|
| add                 | $(c_0 + c'_0, c_1 + c'_1)$                                            | element-wise add                | cheap                        |
| $\times$ plaintext  | $(p c_0, p c_1)$                                                      | element-wise mul                | cheap; then rescale          |
| $\times$ ciphertext | $(c_0 c'_0, c_0 c'_1 + c_1 c'_0, c_1 c'_1)$ , now under $(1, s, s^2)$ | 4 element-wise mul              | cheap, but needs relinearize |
| relinearize         | fold the $s^2$ term back into two components using $rlk$              | <b>key switch</b>               | expensive                    |
| rescale             | divide by $q_\ell$ , drop that limb; $\Delta^2 \rightarrow \Delta$    | INTT/NTT + basis conversion     | moderate; uses 1 level       |
| rotate by $r$       | $X \mapsto X^{5^r}$ shifts the slots, then key switch with $gk_r$     | permutation + <b>key switch</b> | expensive                    |

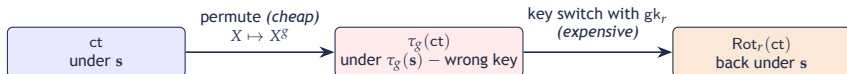
- Everything stays in RNS + NTT form; add and multiply are embarrassingly parallel over the  $2LN$  words.
- A ciphertext multiply is really *multiply*  $\rightarrow$  *relinearize*  $\rightarrow$  *rescale*, and the key switch dominates it.

# Rotation: a cheap permutation plus a key switch

|                     |     |     |     |     |
|---------------------|-----|-----|-----|-----|
| $z$                 | 1.0 | 2.1 | 3.0 | 2.5 |
| $\text{Rot}_1(z)$   | 2.1 | 3.0 | 2.5 | 1.0 |
| $t_1$               | 3.1 | 5.1 | 5.5 | 3.5 |
| $\text{Rot}_2(t_1)$ | 5.5 | 3.5 | 3.1 | 5.1 |
| $t_2$               | 8.6 | 8.6 | 8.6 | 8.6 |

Sum all slots:  $\log_2 4 = 2$  rotate-and-adds.

- $\tau_g : m(X) \mapsto m(X^g)$  with  $g = 5^r \bmod 2N$  shifts the slots by  $r$  (5 generates the cyclic slot order).
- On stored data  $\tau_g$  is just an **index permutation** inside every limb – cheap data movement.
- But the result decrypts under  $\tau_g(s)$ , and ciphertexts under different keys cannot be added  $\Rightarrow$  **key switch** back to  $s$  with  $gk_r$ .
- Each rotation distance needs its own key.

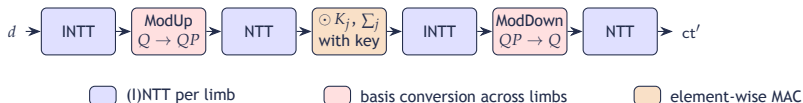


# Key switching: the expensive step

Both leave a term under the “wrong” key,  $d \cdot s_{\text{old}}$  with  $s_{\text{old}} = s^2$  (multiply) or  $\tau_g(s)$  (rotate). The key encrypts  $s_{\text{old}}$  under  $s$  in a larger basis  $QP$ , one pair per digit  $d_j$  of  $d = \sum_j d_j G_j$ :

$$K_{0,j} + K_{1,j} s = P G_j s_{\text{old}} + e_j \pmod{QP},$$
$$(c'_0, c'_1) = \text{ModDown} \left( \sum_j \text{ModUp}(d_j) \cdot (K_{0,j}, K_{1,j}) \right),$$

so  $c'_0 + c'_1 s \approx d s_{\text{old}}$  – the term now lives under  $s$ .



## Why it dominates

Every stage touches all  $L+K$  limbs of every digit, and the key – several ciphertexts’ worth of data – is streamed from memory for each key switch, so it is typically memory-bandwidth bound.



# The cross-limb kernels: ModUp, ModDown, Rescale

Everything else is per limb. In these three, each output residue depends on *all* input residues of the same coefficient (a CRT basis conversion, done in coefficient form).

## ModUp $Q \rightarrow QP$

Add residues modulo new primes  $p_j$  of the *same* integer.

Toy:  $x = 558$ ,  $Q = 17 \cdot 97$ ,  
 $P = 113$ :

$$(14, 73) \mapsto (14, 73, 106)$$

## ModDown $QP \rightarrow Q$

Divide by  $P$  and round:  
 $y = \lfloor X/P \rfloor$ .

Toy:  $X = 113 \cdot 558 + 20$ :

$$(4, 24, 20) \mapsto (14, 73) = 558$$

Just dropping the  $p$  row gives  
 $(4, 24) = 412$  – wrong.

## Rescale $Q_\ell \rightarrow Q_{\ell-1}$

ModDown by a chain prime  
 $q_\ell$ :  $\Delta \rightarrow \Delta/q_\ell$ , one level  
used.

Toy:  $\Delta = 170$ ,  $c = 85$  ( $= 0.5$ ):

$$(0, 85) \mapsto (5)$$

as  $85 \cdot 17^{-1} \equiv 5 \pmod{97}$ ;  
new  $\Delta' = 10$ :  $5/10 = 0.5$ . ✓

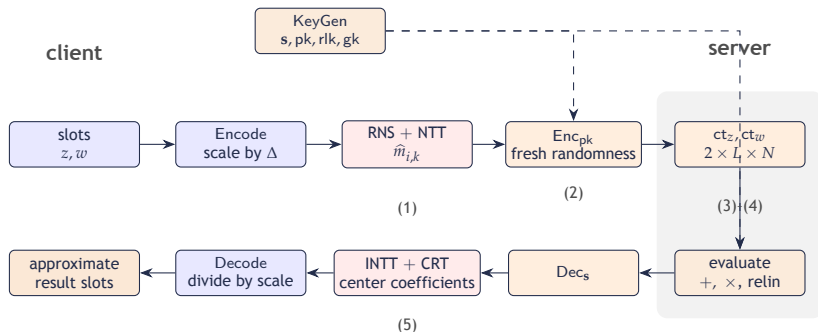
## In hardware terms

Fast basis conversion computes, per coefficient,

$\hat{x}_j = \sum_i [x_i \hat{Q}_i^{-1}]_{q_i} \hat{Q}_i \bmod p_j$  with  $\hat{Q}_i = Q/q_i$ : a small matrix-vector product of modular MACs across limbs, sandwiched between INTT and NTT.

# Example workflow: two vectors, end to end

Toy, insecure parameters:  $N = 8$  (four slots),  $\Delta = 10$ ,  $Q = q_0 q_1 = 17 \cdot 97$  with  $q_i \equiv 1 \pmod{16}$  (NTT-friendly). Inputs  $z = (1.0, 2.1, 3.0, 2.5)$  and  $w = (1.2, 2.2, 3.1, 2.6)$ ; goal:  $z + w$  and  $z \odot w$ .



Each ciphertext is a  $2 \times 2 \times 8$  tensor here. The next slides trace one NTT column ( $k = 0$ ) through both limbs; every other column is computed the same way, independently.

# Workflow 1 – client: encode, split into limbs, NTT

$m_z = \lfloor 10 \sigma^{-1}(z) \rfloor$ , then reduce mod each  $q_i$ , then NTT $_{q_i}$  with primitive 16-th roots  $\psi_0 = 3, \psi_1 = 8$ :

|                   |    |    |    |   |   |    |   |   |
|-------------------|----|----|----|---|---|----|---|---|
| coefficient $j$ : | 0  | 1  | 2  | 3 | 4 | 5  | 6 | 7 |
| $m_z$             | 22 | -4 | -3 | 1 | 0 | -1 | 3 | 4 |
| mod 17            | 5  | 13 | 14 | 1 | 0 | 16 | 3 | 4 |
| mod 97            | 22 | 93 | 94 | 1 | 0 | 96 | 3 | 4 |

↓ NTT, each limb separately

|                      |    |    |    |    |    |    |    |    |
|----------------------|----|----|----|----|----|----|----|----|
| NTT coordinate $k$ : | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| $\hat{m}_z \bmod 17$ | 9  | 9  | 16 | 3  | 3  | 16 | 9  | 9  |
| $\hat{m}_z \bmod 97$ | 41 | 82 | 46 | 16 | 16 | 46 | 82 | 41 |

- Every slot spreads over all eight coefficients; decoding  $m_z/10$  gives  $(1.113, 2.133, 3.115, 2.438) \approx z$  – the tiny  $\Delta$  makes rounding error visible.
- $m_w$  differs only in  $m_0$  (23 vs. 22), so every  $\hat{m}_w = \hat{m}_z + 1$ : 10 and 42 at  $k = 0$ .
- Each row is a limb; the column index  $k$  is an NTT coordinate, not a slot.

## Workflow 2 – client: keys and encryption

Column  $k = 0$  only. Ternary secret  $s(X) = -1 + X - X^4 + X^5 - X^6 - X^7$ , so  $\hat{s} = 2$  in both limbs. Public key  $\text{pk} = (b, a)$  with  $b = -as + e$ ; encrypt with small random  $u$ :

$$\text{ct} = (c_0, c_1) = (m + b u + e_0, a u + e_1), \quad c_0 + c_1 s = m + \text{small}.$$

Toy trace:  $\hat{a} = 5$  and all errors 0 (insecure, but checkable by hand).

| limb       | $\widehat{\text{pk}} = (\hat{b}, \hat{a})$ | $\text{ct}_z: \hat{m}, \hat{u} \rightarrow (c_0, c_1)$ | $\text{ct}_w: \hat{m}, \hat{u} \rightarrow (c_0, c_1)$ | check $c_0 + 2c_1$  |
|------------|--------------------------------------------|--------------------------------------------------------|--------------------------------------------------------|---------------------|
| $q_0 = 17$ | $(7, 5)$                                   | $9, 11 \rightarrow (1, 4)$                             | $10, 4 \rightarrow (4, 3)$                             | $9, 10 \checkmark$  |
| $q_1 = 97$ | $(87, 5)$                                  | $41, 79 \rightarrow (27, 7)$                           | $42, 41 \rightarrow (20, 11)$                          | $41, 42 \checkmark$ |

- $u$  is small in coefficient form; its NTT values (11, 79, ...) need not look small.
- Each ciphertext: 2 components  $\times$  2 limbs  $\times$  8 coordinates = 32 words (real:  $2 \times 30 \times 65,536$ ).

# Workflow 3 – server: add and multiply, cell by cell

With  $\text{ct}_z = (a_0, a_1)$  and  $\text{ct}_w = (b_0, b_1)$ , each  $(\text{limb}, k)$  cell computes independently:

$$\text{ct}_z + \text{ct}_w = (a_0 + b_0, a_1 + b_1),$$

$$\text{ct}_z \cdot \text{ct}_w = (a_0 b_0, a_0 b_1 + a_1 b_0, a_1 b_1) \quad \text{under } (1, s, s^2).$$

|          | cell                          | result         | decrypts to                                                            |
|----------|-------------------------------|----------------|------------------------------------------------------------------------|
| add      | $q_0: (1, 4) + (4, 3)$        | $(5, 7)$       | $5 + 2 \cdot 7 \equiv 2 = 9 + 10 \checkmark$                           |
|          | $q_1: (27, 7) + (20, 11)$     | $(47, 18)$     | $47 + 2 \cdot 18 = 83 = 41 + 42 \checkmark$                            |
| multiply | $q_0: (1, 4) \cdot (4, 3)$    | $(4, 2, 12)$   | $4 + 2 \cdot 2 + 4 \cdot 12 \equiv 5 \equiv 9 \cdot 10 \checkmark$     |
|          | $q_1: (27, 7) \cdot (20, 11)$ | $(55, 49, 77)$ | $55 + 2 \cdot 49 + 4 \cdot 77 \equiv 73 \equiv 41 \cdot 42 \checkmark$ |

- Decoded sum:  $(2.326, 4.367, 6.330, 4.977) \approx z + w = (2.2, 4.3, 6.1, 5.1)$ .
- Product: three components at scale  $\Delta^2 = 100$ ; decodes to  $(1.350, 4.764, 10.016, 6.189) \approx z \odot w = (1.20, 4.62, 9.30, 6.50)$ . Next: relinearize, then rescale.
- Hardware view:  $2LN$  modular adds or  $4LN$  modular multiplies, all independent – pure streaming.

## Workflow 4 – server: relinearize with $\text{rlk}$

Toy one-digit key  $\text{rlk} = (\beta, \alpha)$  with  $\beta + \alpha s = s^2$ . At  $k = 0$ :  $\hat{s}^2 = 4$  and  $\alpha = 5$ , so  $\beta = 4 - 10$ , i.e.  $(11, 5) \bmod 17$  and  $(91, 5) \bmod 97$ . Then

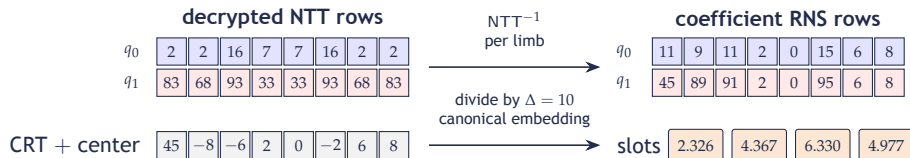
$$(d_0, d_1, d_2) \mapsto (d_0 + d_2\beta, d_1 + d_2\alpha).$$

| cell  | raw product    | $+ d_2 \cdot \text{rlk}$ | two components, decrypt                          |
|-------|----------------|--------------------------|--------------------------------------------------|
| $q_0$ | $(4, 2, 12)$   | $+ 12 \cdot (11, 5)$     | $(0, 11): 0 + 2 \cdot 11 \equiv 5 \checkmark$    |
| $q_1$ | $(55, 49, 77)$ | $+ 77 \cdot (91, 5)$     | $(78, 46): 78 + 2 \cdot 46 \equiv 73 \checkmark$ |

- Same value, same scale  $\Delta^2$ , back to two components. A rescale then drops a limb and returns the scale to  $\approx \Delta$ .
- A real  $\text{rlk}$  is noisy, multi-digit and lives in  $QP$ , so this one line becomes the full key-switch pipeline ( $\text{INTT} \rightarrow \text{ModUp} \rightarrow \text{NTT} \rightarrow \text{MAC} \rightarrow \text{INTT} \rightarrow \text{ModDown} \rightarrow \text{NTT}$ ) – where most of the runtime goes.
- Rotation keys  $\text{gk}_r$  are built the same way, for  $\tau_{5r}(s)$  instead of  $s^2$ .

# Workflow 5 – client: decrypt, INTT, CRT, decode

Decrypt every cell,  $\hat{m} = \hat{c}_0 + \hat{c}_1 \hat{s}$  (e.g. the sum at  $(q_0, k=0)$ :  $5 + 2 \cdot 7 \equiv 2$ ), then undo the representation:



$$\text{Decode}((45, -8, -6, 2, 0, -2, 6, 8)/10) \approx z + w = (2.2, 4.3, 6.1, 5.1).$$

Decrypt, INTT and CRT are exact; the approximation comes from the scaled, rounded encoding (and, in real CKKS, the noise).

# CKKS through an architect's lens

| kernel                     | used by                           | shape                                                                            | usually limited by                  |
|----------------------------|-----------------------------------|----------------------------------------------------------------------------------|-------------------------------------|
| NTT / INTT                 | encode, rescale, every key switch | per limb: $\log_2 N$ stages of $N/2$ butterflies; the stride changes every stage | modular multipliers, data shuffling |
| element-wise mod mul / add | add, multiply, key-switch MAC     | $2LN$ independent lanes                                                          | memory bandwidth                    |
| basis conversion           | ModUp, ModDown, rescale           | per coefficient: small mat-vec across limbs                                      | modular MACs                        |
| automorphism               | rotation                          | fixed index permutation inside each limb                                         | data movement                       |
| key switch                 | relinearize, rotate, bootstrap    | all of the above + a large key                                                   | streaming the key                   |

- The primitive ALU op is a 32/64-bit modular multiply (Barrett / Montgomery).
- Working set:  $\sim 30$  MB per ciphertext, keys several times that – far beyond on-chip SRAM, so data reuse and scheduling matter.
- Key switching dominates multiplication, rotation and bootstrapping – the main target of GPU and ASIC accelerators.



# Table of Contents

- 1 Motivation: Computing on Encrypted Data
- 2 Bootstrapping
- 3 Lattices & Learning With Errors
- 4 Mathematical Background
- 5 The CKKS Scheme
- 6 CKKS on GPUs**

# Why accelerate? FHE is $10^4\times$ slower than plaintext

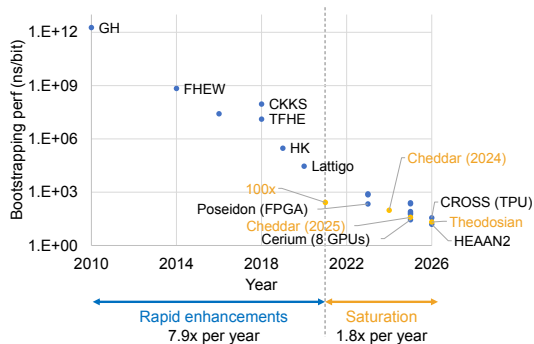
## ResNet-20 inference

plaintext  $< 0.1\text{ s}$

FHE, 1-thread CPU **38 min**

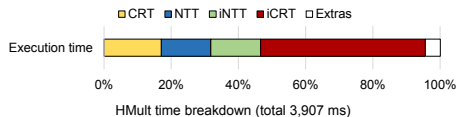
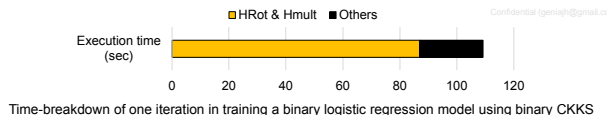
FHE, RTX 5090 **0.57 s**  
(Cheddar)

- $\approx 4000\times$  from the GPU library alone.
- Data blow-up: a 1 MB message becomes a 24 MB ciphertext.
- Each op becomes a chain of NTT / BConv kernels, plus bootstrapping.



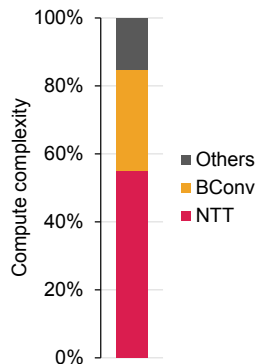
Bootstrapping cost (ns/bit, lower is better):  $7.9\times/\text{year}$   
until  $\sim 2021$ , only  $1.8\times/\text{year}$  since.

# Where the time goes: key switching, NTT and CRT



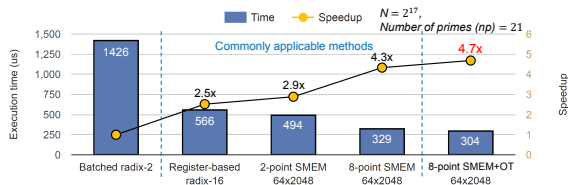
## Binary (non-RNS) CKKS on a CPU

- HRot and HMult – the ops that hide a key switch – take  $\approx 80\%$  of a training iteration.
- Inside HMult, NTT/iNTT and CRT/iCRT take  $\approx 95\%$  of the time; iCRT alone is about half.
- RNS-CKKS keeps the same two kernels: NTT and BConv (the RNS form of CRT), right.

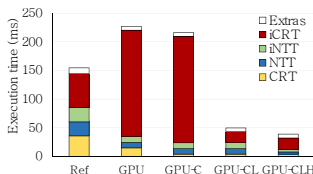
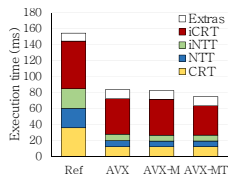


## RNS-CKKS: compute complexity

# First GPU kernels: NTT and HMult

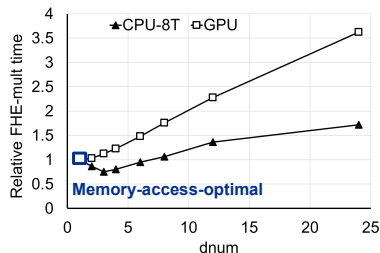


NTT (IISWC'20;  $N = 2^{17}$ , 21 primes): 1426  $\rightarrow$  304  $\mu$ s (4.7 $\times$ ) from high radix, on-the-fly twiddles, register / shared-memory reuse and pipelined transposes.

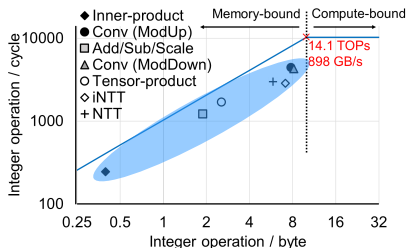


HMult (IEEE Access'21; binary CKKS,  $N = 2^{16}$ ,  $Q = 2^{1200}$ ) vs. a 24-thread CPU: AVX-512 2.06 $\times$ , GPU (Titan RTX) 4.05 $\times$ . A naive GPU port is *slower* – iCRT dominates – until loop reordering (CL) and high-radix NTT (CLH).

# The turning point: CKKS on GPUs is memory-bound



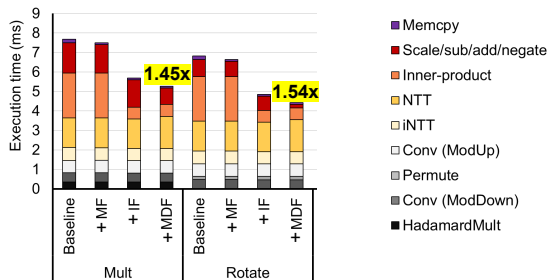
*dnum* vs. multiplication time for a fixed  $Q$



Roofline plot of an HMult

- Right: every HMult kernel sits in the memory-bound region of a V100 (14.1 TOPs, 898 GB/s) – reducing memory access is what matters.
- Left: on the GPU the fastest dnum is the one that minimizes memory traffic, not the one an 8-thread CPU prefers. Prior GPU work used  $dnum = L$ .

# Kernel fusion on GPUs (CHES 2021)



Latency with  $\text{dnum} = 3$  [HK20]

- Each fusion step (MF, IF, MDF) removes a pass of the  $2LN$ -word ciphertext through DRAM.
- HMult  $1.45\times$ , HRot  $1.54\times$  from fusion alone.
- $7.02\times$  faster HMult than the prior GPU implementation, which used  $\text{dnum} = L$ .

$V_{100}$ ,  $N = 2^{16}$ ,  $\log Q = 2305$ ,  
 $L = 44$ .

| boot (ms), $(N, L, \text{dnum})$ | baseline | +fusion | +batching | +hoisting     | vs. CPU                       |
|----------------------------------|----------|---------|-----------|---------------|-------------------------------|
| $(2^{16}, 34, 5)$                | 428.94   | 377.78  | 351.09    | <b>328.25</b> | <b>242<math>\times</math></b> |
| $(2^{17}, 29, 3)$                | 719.87   | 623.92  | 568.20    | <b>526.96</b> | <b>257<math>\times</math></b> |

fusion = intra-HE-op fusion; batching = mult-and-add batching; CPU = single thread.

# GPU libraries today: per-operation latency

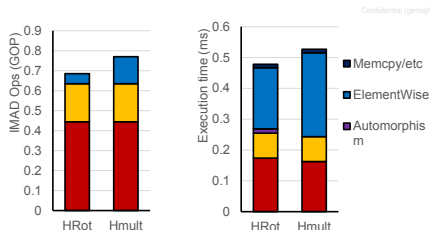
RTX 4090, latency in  $\mu\text{s}$

| library        | limbs     | $\alpha$  | HMult      | HRot       | HAdd      | Rescale   |
|----------------|-----------|-----------|------------|------------|-----------|-----------|
| Liberate.FHE   | 24        | 6         | 5989       | 4554       | 96        | 150       |
| HEonGPU        | 24        | 6         | 2427       | 2279       | 79        | 255       |
| Phantom        | 24        | 6         | 958        | 850        | 83        | 114       |
| <b>Cheddar</b> | <b>48</b> | <b>12</b> | <b>533</b> | <b>476</b> | <b>48</b> | <b>68</b> |
| Liberate.FHE   | 12        | 6         | 2848       | 2052       | 22        | 120       |
| HEonGPU        | 12        | 6         | 825        | 740        | 10        | 113       |
| Phantom        | 12        | 6         | 380        | 336        | 18        | 68        |
| <b>Cheddar</b> | <b>24</b> | <b>12</b> | <b>222</b> | <b>191</b> | <b>11</b> | <b>47</b> |

$\alpha$  = limbs per key-switching digit.

- HMult and HRot hide a key switch:  $\approx 10\text{-}80\times$  the cost of an HAdd.
- Cheddar:  $1.7\text{-}1.8\times$  faster HMult than Phantom,  $11\text{-}13\times$  faster than Liberate.FHE.
- Cheddar uses 32-bit words, so the same modulus needs twice the limbs (48 vs. 24) – and it still wins.

# Compute is NTT-heavy – time is not



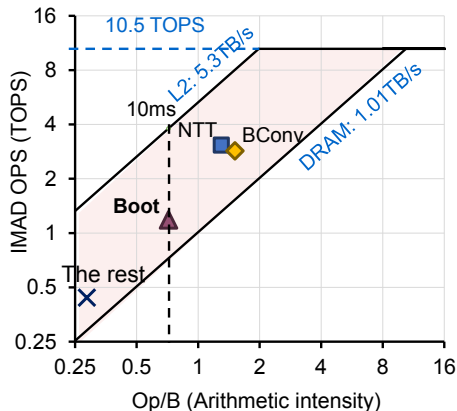
HRot / Hmult on an RTX 4090: 32-bit integer multiply-add (IMAD) ops (left) vs. time (right). Red / yellow = NTT / BConv (legend truncated in the source).

- NTT + BConv do  $> 80\%$  of the integer ops of an HRot or Hmult...
- ...but element-wise ops (blue) take  $\approx 40\text{-}50\%$  of the time: they move the most bytes per op.

| bootstrapping, RTX 4090        | IMAD ops (GOP) | time (ms)  | traffic (GB) | op/B       |
|--------------------------------|----------------|------------|--------------|------------|
| NTT                            | 21 (55%)       | 6.7 (21%)  | 16           | 1.3        |
| BConv                          | 8 (21%)        | 3.0 (9%)   | 5.6          | 1.4        |
| the rest (mostly element-wise) | 9 (24%)        | 20.0 (62%) | 31           | 0.3        |
| memcpy                         | -              | 0.6        | 0.6          | -          |
| <b>total</b>                   | <b>38</b>      | <b>32</b>  | <b>53</b>    | <b>0.7</b> |



# The roofline: CKKS on a GPU is memory-bound



RTX 4090 roofline, bootstrapping kernels

- Ceilings: 10.5 TOPS of 32-bit IMAD; L2 5.3 TB/s; DRAM 1.01 TB/s. Ridge points:  $\approx 2$  op/B (L2) and  $\approx 10$  op/B (DRAM).
- Every CKKS kernel sits left of both: NTT / BConv  $\approx 1.3$ -1.4, the rest  $\approx 0.3$ , bootstrapping as a whole  $\approx 0.7$  op/B.
- Bootstrapping moves 53 GB in  $32 \text{ ms} \approx 1.7 \text{ TB/s}$  – more than DRAM can deliver, so L2 reuse is already essential.
- **Implication:** speedups come from moving fewer bytes (fusion, on-chip reuse, smaller keys), not from more ALUs.

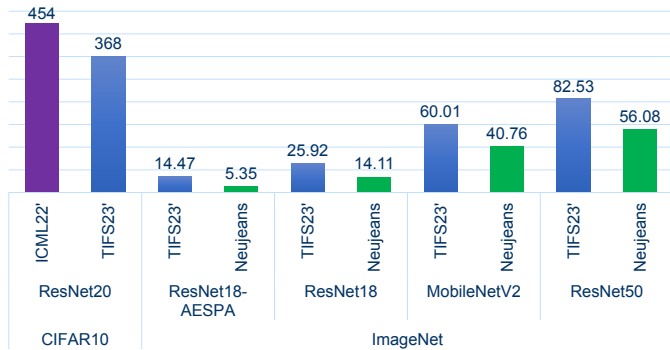
# Bootstrapping latency across platforms

| GPU implementation (hardware, year) | boot (ms)   |
|-------------------------------------|-------------|
| 100× (V100, '21)                    | 328.0       |
| TensorFHE (A100 40GB, '23)          | 250.0       |
| HEaaN GPU (A100 80GB, '23)          | 171.0       |
| WarpDrive (A100 80GB, '25)          | 121.0       |
| FIDESlib (RTX 4090, '25)            | 146.0       |
| Cerium (DGX A100, '25)              | 34.2        |
| Cerium (DGX H100, '25)              | 20.0        |
| Cerium (DGX B200, '25)              | 14.5        |
| <b>Cheddar (A100 80GB)</b>          | <b>40.0</b> |
| <b>Cheddar (H100 80GB)</b>          | <b>31.2</b> |
| <b>Cheddar (RTX 5090)</b>           | <b>22.1</b> |
| <b>Theodosian (A100 80GB)</b>       | <b>31.8</b> |
| <b>Theodosian (H100 80GB)</b>       | <b>23.6</b> |
| <b>Theodosian (RTX 5090)</b>        | <b>15.2</b> |

| FPGA / TPU (year)     | boot (ms) |
|-----------------------|-----------|
| FAB (FPGA, '23)       | 477.0     |
| Poseidon (FPGA, '23)  | 128.0     |
| EFFACT (FPGA, '25)    | 148.0     |
| Cross (TPUv4-8, '26)  | 129.8     |
| Cross (TPUv5e-4, '26) | 59.2      |
| Cross (TPUv5p-8, '26) | 68.3      |
| Cross (TPUv6e-8, '26) | 21.5      |

- One GPU: 328  $\rightarrow$  15 ms in five years ( $\approx 22\times$ ).
- One RTX 5090 (15.2 ms)  $\approx$  an 8-GPU DGX B200 (14.5 ms).
- FPGAs and TPU slices land in the same range or slower.

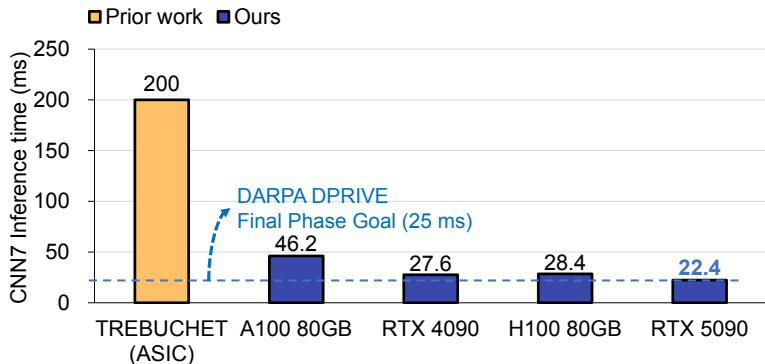
# Algorithms matter too: end-to-end CNN latency



End-to-end inference time (s) as the FHE algorithms improve: ICML'22  $\rightarrow$  TIFS'23  $\rightarrow$  NeuJeans.

- Same models, newer algorithms: 1.5-1.8 $\times$  on ImageNet (ResNet-18 25.9  $\rightarrow$  14.1 s, ResNet-50 82.5  $\rightarrow$  56.1 s).
- Changing the network for FHE (ResNet-18 with AESPA polynomial activations) cuts another 1.8-2.6 $\times$ , down to 5.35 s.

# CNN7: GPUs meet the DARPA DPRIVE target

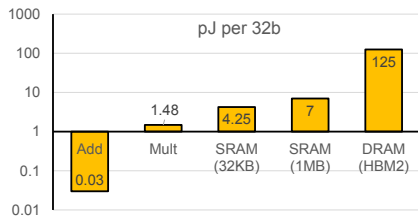


- CNN7 (7-layer CNN, one CIFAR-10 image) is the DARPA DPRIVE milestone. GPUs now beat its 25 ms final-phase goal (RTX 5090: 22.4 ms) and a projected ASIC (TREBUCHET, 200 ms).
- Consumer RTX parts match or beat an H100: FHE makes little use of the tensor-core FLOPS that datacenter GPUs are built around.

# GPU trends vs. what FHE needs

|                       | V100 | A100 | H100 | B200 |
|-----------------------|------|------|------|------|
| INT32 (TOPS)          | 14.1 | 19.5 | 25.6 | -    |
| FP16 tensor (TFLOPS)* | 125  | 312  | 990  | 2250 |
| LLC (MB)              | 6    | 40   | 50   | 126  |
| HBM BW (GB/s)         | 900  | 2039 | 3350 | 8000 |
| HBM capacity (GB)     | 32   | 80   | 80   | 192  |
| TDP (W)               | 300  | 400  | 700  | 1000 |

\*without sparsity; SXM parts.

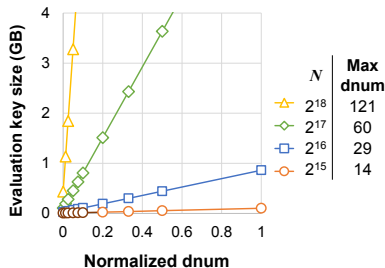


Energy per 32-bit op or access (7 nm)

- The unit FHE needs – 32-bit integer multiply – grew only  $1.8\times$  from V100 to H100; FP16 tensor FLOPS grew  $7.9\times$ , HBM bandwidth  $3.7\times$ .
- A DRAM access costs  $\approx 85\times$  a 32-bit multiply (125 vs. 1.48 pJ): keep data on chip. Growing LLCs (6  $\rightarrow$  126 MB) are what FHE can exploit.

# Takeaways for architects

- GPUs made CKKS practical: bootstrapping in 15-40 ms, ResNet-20 in under a second.
- The bottleneck is **data movement**: kernels run below 2 op/B, and element-wise ops dominate the time.
- **Evaluation keys** are the largest stream – GBs at  $N \geq 2^{17}$  (right) and hard to reuse.
- Levers: kernel fusion, key reuse (hoisting), L2-resident working sets, 32-bit arithmetic, and the choice of dnum.
- Gains have slowed to  $1.8\times$ /year; GPUs still lack large on-chip storage and fast inter-SM shuffles, which motivates FHE ASICs (BTS, ARK, SHARP).



Evaluation-key size vs. normalized dnum (BTS, ISCA 2022)

# References and acknowledgments

## Foundations

- [1] C. Gentry, *A Fully Homomorphic Encryption Scheme*, PhD thesis, Stanford, 2009.
- [2] M. van Dijk, C. Gentry, S. Halevi, V. Vaikuntanathan, “Fully Homomorphic Encryption over the Integers,” EUROCRYPT 2010.
- [3] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” STOC 2005.
- [4] NIST, FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA), 2024.
- [5] J. H. Cheon, A. Kim, M. Kim, Y. Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” ASIACRYPT 2017.
- [6] Microsoft SEAL, CKKS basics example (native/examples/5\_ckks\_basics.cpp).

## Figures and tutorial

- [7] J. H. Ahn, J. Kim, W. Choi, *FHE & Cheddar*, ISCA 2026 tutorial — source of the HE-flow figures and of the GPU-section charts and tables.
- [8] Jayashankar et al., “Cerium: A GPU-Efficient Framework for FHE-Based Inference,” arXiv:2512.11269 — RNS-CKKS figure.

## Acknowledgment

These slides were organized with help from **Claude Code** (Anthropic): restructuring the CKKS section for an architecture audience, the worked NTT / CRT / CKKS examples (every number checked by script), the TikZ figures, the GPU-section charts and tables taken from [7], and the build setup.

## GPUs and accelerators

- [9] S. Kim, W. Jung, J. Park, J. H. Ahn, “Accelerating Number Theoretic Transformations for Bootstrappable HE on GPUs,” IISWC 2020.
- [10] W. Jung et al., “Accelerating FHE Through Architecture-Centric Analysis and Optimization,” IEEE Access 2021.
- [11] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, Y. Lee, “Over 100x Faster Bootstrapping in FHE through Memory-centric Optimization with GPUs,” CHES 2021.
- [12] W. Choi, J. Kim, J. H. Ahn, “Cheddar: A Swift FHE Library Designed for GPU Architectures,” ASPLOS 2026.
- [13] W. Choi et al., “Theodosian: A Deep Dive into Memory-Hierarchy-Centric FHE Acceleration,” arXiv 2026.
- [14] S. Kim et al., “BTS,” ISCA 2022; J. Kim et al., “ARK,” MICRO 2022; J. Kim et al., “SHARP,” ISCA 2023.
- [15] E. Lee et al., “Low-Complexity Deep Convolutional Neural Networks on FHE Using Multiplexed Parallel Convolutions,” ICML 2022.
- [16] J. H. Ju et al., “NeuJeans: Private Neural Network Inference with Joint Optimization of Convolution and FHE Bootstrapping,” CCS 2024.
- [17] D. B. Cousins et al., “TREBUCHET Fully Homomorphic Encryption Accelerator: Phase Two Performance Estimation Results,” GOMACTech 2024.
- [18] N. P. Jouppi et al., “Ten Lessons From Three Generations Shaped Google’s TPUv4i,” ISCA 2021.